Using Private Link for Cost Management API Calls

Purpose and support status

This guide describes the changes needed to evaluate and configure private connectivity from the Azure FinOps solution's Azure Function to the Azure Cost Management API in **Azure Commercial**.

The proposed path uses **Azure Resource Manager (ARM) Private Link**, not a dedicated Cost Management private endpoint. The current solution does not deploy this ARM Private Link configuration.

> **Prerequisite: confirm API compatibility before implementation.** Microsoft documents ARM Private Link, but the general documentation does not explicitly confirm the exact Cost Management operation used by this solution. Obtain confirmation from Microsoft for the operation, API version, and subscription scopes below, then validate them in an approved test environment. This guide is conditional implementation guidance, not a statement that the integration has been proven to work.

The solution currently sends:

```
POST https://management.azure.com/subscriptions/<subscription-id>/providers/
Microsoft.CostManagement/query?api-version=2023-11-01
```

It also reads subscription metadata through ARM. Include those calls in compatibility testing. If billing-audit collection is enabled, validate its ARM operations as well.

What this addresses

The Function currently initiates outbound HTTPS connections to Microsoft APIs. These calls **do not require an inbound public port to be opened** on the Function or the customer's network.

Private Link addresses a different requirement: accessing ARM through a private endpoint IP rather than its public endpoint. Managed identity continues to provide authentication; Private Link provides the network path. Neither replaces the other.

Connection	Effect of this change
Function to Cost Management through ARM	Intended to use the ARM private endpoint, subject to compatibility validation.
Function to PAYG Retail Prices (`prices.azure.com`)	Remains public outbound HTTPS. No customer-configurable Private Link option is documented for this API.

Connection	Effect of this change
Function to SQL, Storage, Key Vault, and monitoring	Not automatically privatized by ARM Private Link. Each has its own networking requirements.
Clients or deployment tools connecting to the Function	Controlled separately by the Function's inbound access configuration.

This change alone does **not** make the entire solution private-only.

Prerequisites and design gates

Requirement	Action
Confirmed API support	Confirm the exact Cost Management query and supporting ARM calls with Microsoft.
Tenant scope	ARM Private Link associations are configured at the tenant's root management group. Obtain platform-team approval; this is not just a Function-level setting.
Setup permissions	Microsoft lists Owner on the subscription hosting the resource-management private link and Owner or Contributor at the root management group. Use a controlled deployment identity; do not grant these permissions to the Function.
Function hosting plan	Use a plan supporting outbound VNet integration. For the current Bicep design, `EP1` is a suitable option; classic `Y1` does not support this integration. Review any required hosting-plan migration separately.
Network	Provide a delegated Function integration subnet and a separate private-endpoint subnet, with routing and HTTPS connectivity between them.
DNS	Provide private resolution for ARM from the Function's execution environment, including forwarding where custom DNS is used.
Runtime authorization	Keep the Function's managed identity and grant Cost Management Reader at the subscriptions being queried.
Administrative access	Preserve a working deployment and recovery path before changing DNS or restricting access.

Critical: Azure Bastion compatibility

Microsoft explicitly warns that ARM Private Link DNS can cause Azure Bastion to stop working because of the overlap with `management.azure.com`.

The current Bicep solution can deploy Bastion in the same VNet as the Function integration. **Do not simply attach the ARM private DNS zone to that shared network.**

Have the platform/network team approve a supported network and DNS design that preserves Bastion. A separately scoped workload network may need to be evaluated, including its DNS forwarding and access to existing private SQL and Storage endpoints. Network separation alone is not proof that the DNS conflict is resolved.

If Bastion is required and a supported design has not been established, stop before rollout.

Implementation steps

1. Record the existing deployment

Identify the tenant, root management group, target subscriptions, Function plan, integration subnet, DNS servers, private-endpoint subnet, and any Bastion dependency.

Check for existing resource-management private links, associations, and private DNS zones. Reuse approved shared resources rather than creating competing resources or duplicate DNS ownership.

Record the existing configuration and a rollback procedure. Perform the initial deployment in a controlled test environment.

2. Enable outbound VNet integration on the Function

Configure the Function to use the approved delegated integration subnet. This gives the Function outbound connectivity to private endpoints.

For the existing Bicep baseline, these parameters enable its networking foundation:

```
functionPlanSkuName=EP1
enablePrivateNetworking=true
```

These settings do not create an ARM private endpoint. They also enable other private-networking changes, including disabling public access to the Function, SQL, and runtime Storage. Review the full deployment changes and ensure deployment agents and administrators retain connectivity.

If the approved design requires a different VNet to preserve Bastion, adapt the template to use that network rather than enabling the shared-network design unchanged.

For Terraform, add outbound VNet integration and the approved subnet inputs; the current Terraform baseline does not include that wiring.

A private endpoint on the Function itself is an **inbound** control. It does not enable private outbound Cost Management calls.

3. Create or reuse the resource-management private link

Have the platform team perform this separately from ordinary Function deployments:

1. In the Azure portal, search for **Resource management private links**.
2. Reuse an approved resource if available; otherwise select **Create resource management private link**.

3. Select the intended tenant, subscription, resource group, and supported location.
4. Review the tenant/root-management-group scope before creating the resource. Microsoft's documented portal workflow also creates the root-management-group association.
5. Record the resource ID and verify the association.

The resource ID has this form:

```
/subscriptions/<hosting-subscription-id>/resourceGroups/<resource-group>/providers/
Microsoft.Authorization/resourceManagementPrivateLinks/<name>
```

For automated deployment, create the resource and association in a separate, explicitly invoked platform deployment. The association's resource name must be a GUID. Follow Microsoft's command reference rather than adding root-management-group permissions to the routine workload deployment.

Microsoft documents multi-tenant limitations for this feature. Do not assume this association covers collection across different tenants.

4. Create the ARM private endpoint

In **Private Link Center**, create a private endpoint with these settings:

Setting	Value
Resource type	`Microsoft.Authorization/resourceManagementPrivateLinks`
Target resource	The approved resource-management private link
Target subresource / group ID	`ResourceManagement`
Network	The approved workload VNet, or a privately reachable network
Subnet	The private-endpoint subnet, not the delegated Function integration subnet

Ensure the endpoint connection is **Approved**. Record its allocated private IP address or addresses.

Allow outbound TCP 443 from the Function integration subnet to that endpoint. Review NSGs, routes, peering, and firewall rules as applicable. NAT Gateway does not create this private path.

5. Configure private DNS

Use the private DNS zone **`privatelink.azure.com`**. Microsoft's setup guide specifies a **`management`** record pointing to the resource-management private endpoint's private IP.

Associate the endpoint with the zone through a private DNS zone group and expose the zone only to the networks/resolvers approved in the Bastion-safe design. Configure forwarding if the Function uses custom DNS.

From the Function execution environment, confirm that **`management.azure.com`** **ultimately resolves to the expected endpoint private IP**.

Keep the API URL as `https://management.azure.com/...`. Do not replace the application URL with a private IP or a `privatelink` hostname, and do not disable certificate validation.

6. Keep application authentication unchanged

Retain these settings:

```
FINOPS_EXTRACTION_MODE=Native
FINOPS_AUTH_MODE=ManagedIdentity
FINOPS_AZURE_CLOUD=AzureCloud
```

No change to the standard ARM token audience or Cost Management query logic should be necessary. The current `scripts/FinOpsNativeAuth.psm1` already uses managed identity and the normal ARM hostname.

Private connectivity does not grant data access. The Function identity still requires Cost Management Reader at the relevant scopes, plus any separately required billing permissions.

7. Enforce the network requirement

Creating a private endpoint establishes a private path; it does not, by itself, prevent public ARM access.

Microsoft currently documents that the root-management-group association's `publicNetworkAccess` field cannot be disabled. If the policy requires this Function to use only the private ARM path, the platform team must enforce that through its outbound network controls.

Design rules that allow the approved private endpoint while denying the Function's public ARM path. An FQDN allow rule for `management.azure.com` alone is not proof of private-path enforcement. Preserve required runtime dependencies and the approved public retail-pricing exception.

Do not block all public egress indiscriminately: retail lookups, telemetry, deployment, and runtime dependencies may need separate access.

Persist the changes in infrastructure as code

Use the portal for an approved proof of concept if appropriate, then reconcile the configuration into the customer's deployment system before production. Do not leave portal changes that a later deployment would overwrite.

Files	Required adaptation
`infra/bicep/main.bicep` and `main.parameters.example.json`	Add opt-in ARM Private Link ID, private endpoint, DNS zone-group/link or existing-zone inputs, and approved network integration options. Preserve the template's inline subnet declarations.
`infra/terraform/main.tf`, `variables.tf`, `outputs.tf`, and `terraform.tfvars.example`	Add opt-in Function VNet integration, ARM private endpoint, approved network/DNS inputs, validation, and endpoint outputs.

Files	Required adaptation
Separate platform deployment	Manage the resource-management private link and root-management-group association independently of workload deployments.
Infrastructure documentation	Record permissions, DNS/Bastion restrictions, egress enforcement, validation evidence, and rollback.

Keep the feature disabled by default and reject incompatible plans or incomplete network inputs when enabled. Use either Bicep or Terraform to own a given resource, not both.

Terraform currently uses AzureRM `~> 3.115`; verify resource support before implementation. No general provider upgrade or migration of unrelated services is required by this guide.

Validation and acceptance

Perform validation from the **Function execution environment**, not only from an administrator's laptop or a jump box.

Check	Required evidence
DNS	`management.azure.com` resolves to the approved ARM private-endpoint IP addresses.
Endpoint	Connection is Approved and TCP 443 is reachable from the Function.
Authentication	The Function uses its managed identity and retains the required role assignments.
Exact API	The actual subscription-scoped Cost Management query succeeds for a bounded date range and each intended subscription.
Private transport	Network evidence confirms the connection destination is the private endpoint. Repeat the query with the Function's public ARM egress denied in a controlled test. DNS/API success alone is insufficient.
Supporting operations	Subscription metadata and any enabled billing collection continue to work.
Existing services	SQL loads, runtime Storage access, retail lookups, telemetry, and deployment access remain functional.
Bastion	Required Bastion administrative access continues to work after the DNS changes.
End-to-end run	A scheduled ingestion run completes and expected data reaches the durable store.

An HTTP 401 or 403 is not a successful connectivity test for this requirement: the actual authorized query must succeed. Do not record access tokens or customer cost rows in shared diagnostic evidence.

Do not claim private-only Cost Management operation until both the application result and private transport have been verified.

Rollback

Preserve an administrative access path throughout the change. If validation fails, pause the affected ingestion schedule and revert only the workload-specific routing, DNS, endpoint, and integration changes introduced for this feature using the recorded configuration.

Restore public ARM connectivity only if customer policy explicitly permits it; otherwise leave ingestion paused and report the blocker. Do not silently fall back to public access.

Do not delete a shared resource-management private link, root-management-group association, or shared DNS zone as part of a workload rollback without platform-owner approval. Revalidate Bastion and existing SQL/Storage connectivity.

Expected effort

This is primarily an infrastructure and platform-governance change, not an ingestion-code rewrite. Bicep already has much of the networking foundation; Terraform needs more networking configuration. The principal uncertainties are exact API compatibility, tenant-level approval, and a DNS design that preserves Bastion.

If Microsoft does not confirm the required API support, or a supported Bastion-compatible design cannot be established, do not proceed with this approach.

Microsoft references

- Create a private link for managing resources using the Azure portal
- Manage resources through Private Link: commands and limitations
- Azure Functions networking options
- Azure Retail Prices REST API